

Task Discovery Service (TDS): A Dual-Architecture Service Registry for Distributed Systems

Final Year Project Report

February 2026

1 Introduction and Motivation

In large distributed systems, one weak point can force the whole platform to slow down: static service routing. The core issue in the system I worked on at Cubic Transportation Systems was not input handling, but rigid, hard-coded output endpoints. Although the component accepted requests from a dynamic number of devices, every routing change required redevelopment and full regression testing. This made scaling expensive and slow, and over time the original resolvers had to be replaced with load balancers as demand outgrew the fixed design. This project began from that operational bottleneck and asks how service discovery can be made both dynamic and scalable.

1.1 Problem Statement

The problem lay in two key aspects

1. The component was not dynamic even though it performed the same function for any given request from the devices beneath it.
2. The server was mainly overloaded because it accepted the data from the requesting device and passed it on to the resolver. The data was never operated upon or even observed by the component.

I proposed to address these issues by designing an architecture that would perform and scale with the system.

1.2 Research Objectives

This project addresses the service discovery problem by developing the Task Discovery Service (TDS), a flexible registry system that supports both centralised and peer-to-peer (P2P) operational modes within a unified architecture. The primary objectives are:

1. Design and implement a dual-mode service registry supporting both centralised and P2P architectures
2. Develop efficient algorithms for service registration, query routing, and load balancing

3. Create a comprehensive monitoring interface for real-time system visualisation
4. Evaluate performance characteristics under various network conditions and load patterns
5. Provide multiple transport and security options including UDP, TCP, and mutual TLS authentication

1.3 Contribution

The main contribution of this work is a production-ready service discovery system that eliminates the operational mode dichotomy present in existing solutions. TDS allows development teams to begin with a simple centralised deployment and seamlessly transition to distributed P2P operation as scale requirements increase, without changing client code or protocol definitions. This architectural flexibility, combined with comprehensive security features and real-time monitoring capabilities, represents a significant advancement in practical service discovery systems.

2 Project Background and Context

2.1 Service Discovery in Distributed Systems

Service discovery allows one part of a distributed system to find another at runtime. Instead of hard-coding IP addresses or ports, services register themselves and clients query for the address they need. This matters because modern systems change constantly: instances are restarted, scaled up, moved, or fail. If the discovery mechanism is out of date, the rest of the system may fail even when the application logic is correct.

Most existing solutions fall into three broad groups: centralised registries, decentralised peer-to-peer systems, and hybrid systems that combine a registry with programmable traffic control. All three solve the same basic problem, each having their different advantages and drawbacks.

2.1.1 Centralised registry approaches

Centralised approaches keep a single authoritative view of the system. Services register with a central catalogue and clients query that catalogue through an API or DNS interface. Systems such as etcd and other Raft-based control planes are designed to give fresh, strongly consistent control-plane state [3,13,20]. This makes them easier to reason about because there is one place where the truth lives.

Many real systems use this model. Kubernetes keeps a stable service identity while its EndpointSlices track changing pods, and cluster DNS translates the service name to current backends [16]. Consul follows a similar pattern, but adds health checks and multiple discovery methods [2,19].

The main strength of centralised discovery is clarity. It is usually easier to manage, observe, and secure. The main weakness is dependency on the control plane itself. If the registry becomes overloaded or unavailable, discovery becomes a bottleneck.

2.1.2 Decentralised approaches

Decentralised approaches remove the single authority and spread responsibility across many peers. In this family, lookup data is distributed around the network rather than stored in one registry. Systems such as Chord and Kademlia showed that this can scale well by placing data according to a hash of the key, in this case a service name, and routing queries through the network efficiently [1, 10]. Dynamo extended this style of thinking by favouring availability and replication even when the network is unstable [11].

These systems also need a way to detect which peers are alive. Protocols such as SWIM address this through scalable membership and failure detection [12]. At smaller scale, mDNS and DNS-SD provide a simpler decentralised model for local networks, where devices can advertise and discover services without a dedicated server [14, 15].

The strength of decentralised discovery is that there is no single point of failure. The weakness is that it is harder to guarantee that every node has the same up-to-date view of the system. As the network grows, correctness depends on how well peer information spreads.

2.1.3 Traffic-management, load-balancing, and policy-enforcement approaches

A third family focuses less on the problem of *finding* services and more on the problem of *choosing* between available instances and *controlling* which connections are allowed. These approaches usually sit on top of an existing discovery mechanism and add traffic-management or policy-enforcement behaviour.

For load balancing, systems such as Consul and Istio use discovery information to route traffic only to healthy instances and to apply routing strategies beyond simple name resolution [18, 19]. Consul can expose healthy endpoints through DNS and integrate with external load balancers, while Istio applies policies such as least-requests, round-robin, weighted routing, retries, timeouts, and circuit breaking through proxies. This makes them useful when the main challenge is not just locating a service, but distributing load safely and efficiently across many replicas.

For network control, platforms such as Kubernetes provide policy layers that restrict which workloads may communicate at all [17]. NetworkPolicy rules can isolate ingress and egress traffic and allow only selected pods, namespaces, ports, or IP ranges. In effect, these systems act like structured, application-aware firewall mechanisms inside a distributed platform.

The advantage of this family is control. It gives operators a way to combine service discovery with health-aware routing, traffic shaping, and access restrictions. The disadvantage is that it depends on additional infrastructure and configuration. These solutions are powerful, but they can become difficult to reason about because discovery, load balancing, and policy enforcement are spread across several layers rather than handled in one place.

2.2 Security, Reachability, and Policy Correctness

Even with correct discovery and load balancing, communication can still fail if policy blocks the connection. This is why service discovery must be considered alongside reachability and access control.

The firewall literature is useful here. Firewall Policy Queries focuses on detecting rule anomalies such as shadowing and redundancy, while Model Checking Firewall Policy Configurations focuses on verifying that policy behaves as intended [21, 22]. These works reinforce a simple

point: a discovery system is only fully useful when the endpoints it returns are not just correct, but also reachable under the active security policy.

2.3 How TDS Differs, and Its Strengths and Weaknesses

Existing solutions usually require an early commitment to one style of architecture. A team must choose a centralised registry, a decentralised overlay, or a more complex hybrid platform and then build around that choice. TDS differs by supporting both centralised and peer-to-peer operation behind the same client-facing interface.

The main design idea is that services do not interact directly with the registry implementation. Instead, they talk to a local client proxy using the same REGISTER and QUERY operations in both modes. This means the deployment architecture can change without forcing application code to change with it. In practical terms, a system can begin with centralised discovery, which is simpler to operate and easier to observe, and later move to P2P operation if resilience or removal of a central dependency becomes more important.

This gives TDS several inherent strengths. First, it reduces lock-in to a single discovery architecture. Second, it lowers adoption cost because the service-facing interface remains small and stable. Third, it allows the centralised mode to benefit from straightforward control and governance, while the P2P mode can offer better tolerance of single-node failure. Finally, the design leaves room for policy-aware routing, which is important in environments where not every discovered service should be reachable by every requester.

However, this flexibility also introduces weaknesses. Supporting two modes increases implementation complexity because both architectural paths must be maintained and tested. The centralised mode still inherits the usual problem of depending on a core registry service. The P2P mode avoids that single dependency, but it is naturally weaker in terms of global consistency and can become harder to reason about as peer knowledge diverges. In other words, TDS avoids early architectural lock-in, but it does not remove the underlying trade-offs between centralised and decentralised discovery; it exposes both and gives the user a choice.

For that reason, TDS is best understood not as a replacement for every mature discovery platform, but as a practical bridge between architectural extremes. Its contribution is to make movement between those extremes possible without changing the service-side contract.

3 Design

3.1 services, tasks, and the client proxy

The diagram below gives an overview of a client machine in this system. With multiple services registering and querying services to the client proxy which is the local endpoint of this solution.

In this architecture, services running on a machine will query their local instance of the client proxy. As far as the services are concerned, that is all that needs to be done for them to advertise their service along with sending periodic heartbeats. The client proxy will also handle any requests these services have for where to send their data once they have finished the scope of their operations on it. The client proxy is the interface behind which all the mechanisms of this project lie.

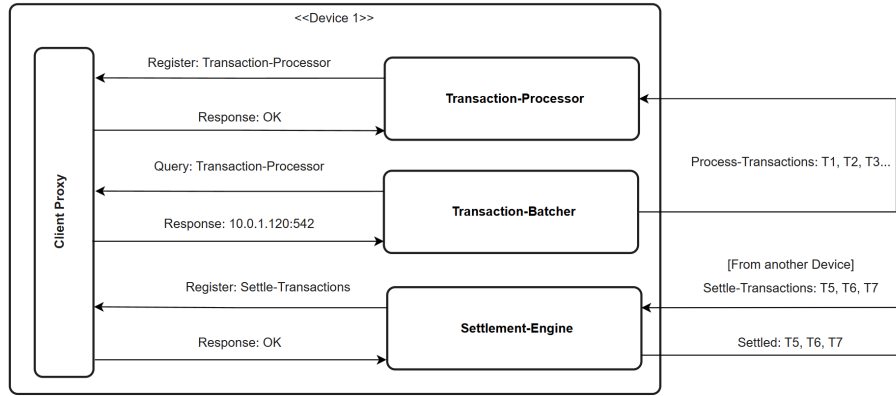


Figure 1: Client machine service registration and discovery via client proxy

3.1.1 Message format

All communication between local services (clients) and the client proxy uses JSON messages in both centralised and P2P modes. The two shared commands are **REGISTER** and **QUERY**.

JSON requests (service → client proxy)

```
{ "cmd": "REGISTER", "task": "payment", "address": "10.0.1.5:8080", "capacity": 10 }
{ "cmd": "QUERY", "task": "payment" }
```

JSON response format

```
{ "status": "OK" } //Register - OK
{ "status": "OK", "address": "10.0.1.5:8080" } //Successful query
{ "status": "NOTFOUND" } // queried task not found
{ "status": "ERR", "error": "task required" } //example error: no task in query request
```

These message types are intentionally minimal so the same client-side behaviour works regardless of whether the proxy is currently forwarding to a central server or to the P2P registry.

3.2 Centralised mode

We now move outside the scope of the single machine registering and querying services to a distributed system containing many such machines. The below figure demonstrates the role the server plays in centralised mode.

3.2.1 load balancing

The server load balances services using a round-robin mechanism for all services under a task. This ensures even distribution of traffic to each service. Configurable on top of this is a weighted round robin mechanism using each registered service's capacity as a weighting. This ensures that services running on less powerful machines are not swamped by being routed the same volume of traffic as a more powerful variant. During the heartbeat message, a service advertises its current capacity to inform load balancing.

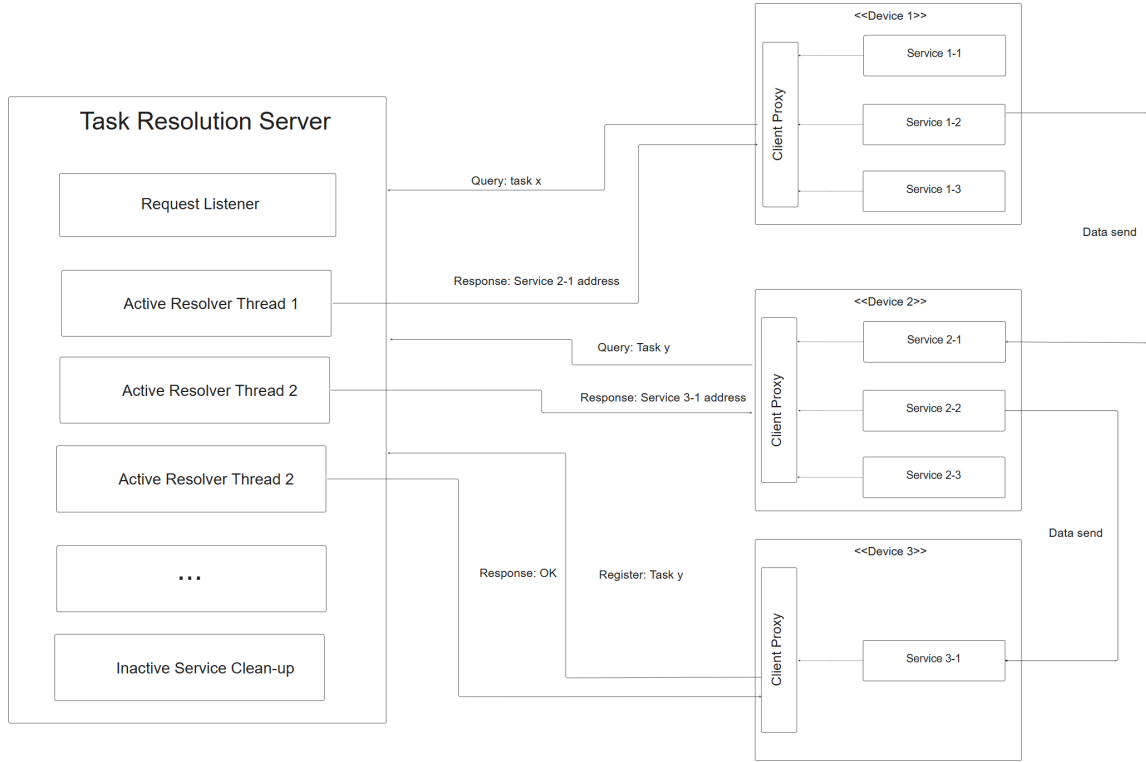


Figure 2: Client machines operating in server architecture

3.2.2 Firewall-mode

In many real use cases of this project, while all machines are able to query and register with the TDS, not all services are allowed to communicate with each other. This is enforced by networking rules that are created and managed by a central authority in the system. The purpose of firewall-mode is to cross-reference a query for a service with the list of ip-address:ports that the requestor can communicate with. This way the server will only return a service address that the client can use.

3.3 Peer-to-Peer mode

The diagram below gives an overview of the peer to peer architecture. It shows how tasks are registered and then stored on the correct node in the network. It also demonstrates service queries and then the separate sending of data outside of the architecture for the process. In contrast to the centralised mode, all functionality of this architecture runs from the client-proxy. The P2P network is formed from one "bootstrap" node. All subsequent nodes that join are given a pre-existing node in the network as a bootstrap. When a node joins, it hashes its ip address to receive their node ID. This ID is capped at 256 bits forming a ring network.

3.3.1 Distributed Hash Table (DHT)

The DHT is used to store the task:address mappings in this architecture. When a service registers a task with their client-proxy, the proxy hashes the service name. It then searches its list of peers to find the node ID that is closest to this hash. Once that node is found the client-proxy sends a STORE request to it with the service details. The node that receives this store

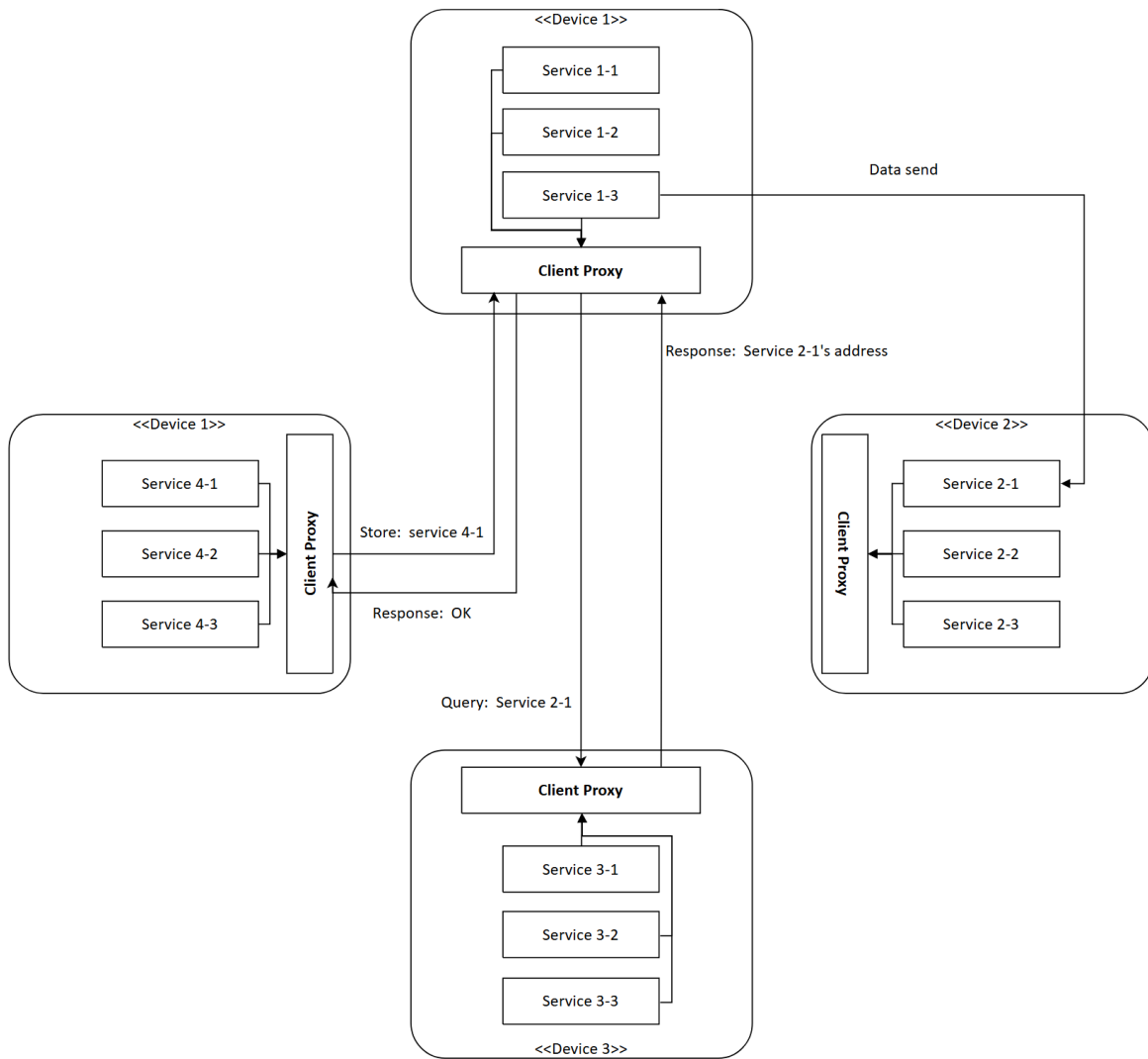


Figure 3: Client machines operating in P2P architecture

message compares its peer list to the task hash to ensure it is the most appropriate candidate. If so then it stores the task in memory similarly to the cache in centralised mode.

A query follows the same path: the client-proxy hashes the query task, sends a query request to the node with the closest ID. If that node has that service stored it returns an address, if not then it returns NOTFOUND.

3.3.2 Eventual consistency and node discovery

The nature of the Distributed Hash Table makes it eventually consistent. Nodes are given an initial peer list from their bootstrap node. As more nodes join the network they discover them through store and query messages as, if they do not have the requesting node in their peer list, they add these nodes to their peer list. Additionally, to supplant this, a periodic re-discovery process runs where nodes query their original bootstrap node for any peers that joined after them. This introduces extra computation onto bootstrap nodes but this can be balanced by carefully choosing more computationally powerful devices in the network to be bootstrap nodes.

3.3.3 K-Nearest Neighbours

On top of the previously discussed node discovery mechanisms, there are protections against the incorrect storage of tasks when naive nodes have not yet discovered all their peers. In the event that a node fails to select the correct node for a given task hash, usually because the correct node is not yet in their peer list, a node will reject a received store request if it does not believe itself to be the closest to the hash and tell the requestor the node they believe is responsible to proactively bridge the knowledge gap. This, however, does not eliminate the possibility of incorrectly storing a task as both nodes might not have the best candidate in their peer list. The implemented protection against this is to implement k-nearest-neighbours: the client-proxy finds the k closest nodes to their hash and sends a store request to all three. As well as providing protection against tasks not being stored correctly it allows for redundant storage of task mappings in the network in case of node loss. With this enabled, nodes will instead check if they are in the best k candidates for the hash instead.

4 Implementation details and testing

4.1 Implementation Challenges

Here i will lay out key insights into areas of my project that presented key challenges to me. It details how I overcame them and, in particular, where I decided to cease further work in the scope of the project timeframe.

4.1.1 Server Cache Performance

The implementation of the cache appeared simple but produced several challenges. Having completed the cache implementation, I ran performance tests with a combination of cache hits and misses. Initially I found that cache misses were actually faster than hits! After increasingly deep investigation I found that, though queries should be read intensive, because they incremented query counts and round robin it was write locking significantly. My solution was two-fold: firstly, I batched query count updates to the database as they were not necessary until the next

cache warming. I then researched in to Atomics for the round robin and with these implemented the cache became significantly faster than the database.

4.1.2 Peer-to-Peer Challenges

Having implemented and tested the P2P mode for this project, I found that it struggled to scale with large numbers of nodes. The issue pertained to the fact that, as the network grew, patterns of individual nodes communicating emerged. Many nodes were rarely contacted and therefore their routing tables were not updated with the latest information. In these cases the centralised approach was far more effective. What the network gained in robustness against node failure, it lost with scale.

4.1.3 Firewall-Mode

Firewall mode was a feature I implemented with the goal of allowing the TDS system (in centralised mode) to only serve up services that were routable for the requestor. This proved to be a significant challenge as the only method I devised was to import firewall rule file from the system administrator of any given application. This was deemed unacceptable as the separation of the TDS' routing table from the source of truth (the firewall) was a recipe for inconsistency. The feature was therefore not developed further but included as an artifact as further work could be done to improve the implementation.

5 Functional testing

6 Realistic Application Testing

To demonstrate the practical utility of TDS, I implemented a mock-up of the system that inspired it. I created a highly abstracted version of a train ticketing, Oyster and contactless payment system.

The system specified can be found under the Simulation folder of the project repository.

It contains a set of programs that simulate a full distributed system with stations, payment gates, fare calculation and very rough PCTR-tokenisation to simulate cardholder environment structure. Key features include:

- Station computers to batch transactions
- Payment gates to process card and ticket presentations
- Fare calculation service to compute trip costs
- Multiple endpoints for ticket distribution to demonstrate load balancing

A full High-Level-Design document is available in the repository under `ticketing_system_High_Level_Design`

All of these devices were interconnected by querying the TDS system for service discovery. Of key note was that, although many issues arose developing this system and setting up the environment, communication between devices was seamless. This was a testament to the robustness of the project as the only time I had to troubleshoot communication was when i forgot to start the TDS server itself.

7 Conclusion and future work

8 Future Work

Several enhancements would improve TDS capabilities:

Replication: Add multi-master replication for centralised mode high availability. Raft consensus could provide strong consistency while tolerating failures.

Health checking: Implement active health probes instead of passive heartbeats. Server could periodically verify service reachability and automatically deregister failed instances.

Service metadata: Support arbitrary key-value tags for services enabling filtering queries (e.g., "version=2.0", "data-centre=us-east").

Multi-data-centre: Extend P2P mode with geographic awareness for locality-based routing.

Metrics export: Integrate with Prometheus for comprehensive monitoring and alerting.

REST API: Add HTTP REST interface alongside binary protocol for broader language support and web-based tooling.

8.1 Conclusion

In summation, this project has achieved the goals outline in project objectives. It provides a versatile achitecture that provides a tradeoff between correctness and decentralisation. The client proxy is a stable interface to all services running on a device enabling complete transparency to the task discovery architecture in place. With well established query, registry and reponse formats it can be easily integrated to any service. the Client package provides functions that can replace any previously existing hard coded endpoint with a call.

Centralised mode offers the benefits of a dedicated device for task discovery. In instances with generous amounts of memory, it can be run entirely from memory without disadvantage as all data must be kept fresh through heartbeats so application restarts cause minimal disruption. In conjunction with a database the server retains functionality with greater heartbeat timeouts and broad task space with a high performance cache that keeps highly queried data for fast access. The performance of these operating modes have been tested and the benefits are noticeable. The firewall functionality ensures that responses to queries are usable by a service according to a set of provided routing rules. As discussed in the implementation section, this project does not aim to replace network security management so the routing is a best effort mechanism from a set of imported rules. A further key note is that, in hybrid mode, since both the firewall rules and tasks are stored in databases, multiple instances of this server can be present on a network to reduce load on a single server but it introduces some inconsistency with cached tasks data. Since all register operations are sent directly to the database, the impact is minimal on the correct listing of tasks.

References

- [1] Stoica, I., Morris, R., Karger, D., Kaashoek, M. F., & Balakrishnan, H. (2001). *Chord: A scalable peer-to-peer lookup service for internet applications*. ACM SIGCOMM Computer Communication Review, 31(4), 149-160.
- [2] HashiCorp. (2021). *Consul: Service Mesh for Microservices*. <https://www.consul.io/>
- [3] CoreOS. (2020). *etcd: A distributed, reliable key-value store*. <https://etcd.io/>
- [4] Hunt, P., Konar, M., Junqueira, F. P., & Reed, B. (2010). *ZooKeeper: Wait-free coordination for internet-scale systems*. USENIX Annual Technical Conference, 10(8), 9.
- [5] Karger, D., Lehman, E., Leighton, T., Panigrahy, R., Levine, M., & Lewin, D. (1997). *Consistent hashing and random trees: Distributed caching protocols for relieving hot spots on the World Wide Web*. ACM Symposium on Theory of Computing, 654-663.
- [6] Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.
- [7] Donovan, A. A., & Kernighan, B. W. (2015). *The Go Programming Language*. Addison-Wesley Professional.
- [8] Cardellini, V., Colajanni, M., & Yu, P. S. (1999). *Dynamic load balancing on web-server systems*. IEEE Internet Computing, 3(3), 28-39.
- [9] Ratnasamy, S., Francis, P., Handley, M., Karp, R., & Shenker, S. (2001). *A scalable content-addressable network*. ACM SIGCOMM Computer Communication Review, 31(4), 161-172.
- [10] Maymounkov, P., & Mazieres, D. (2002). *Kademlia: A peer-to-peer information system based on the XOR metric*. International Workshop on Peer-to-Peer Systems (IPTPS), 53-65.
- [11] DeCandia, G., Hastorun, D., Jampani, M., Kakulapati, G., Lakshman, A., Pilchin, A., Sivasubramanian, S., Voshall, P., & Vogels, W. (2007). *Dynamo: Amazon's highly available key-value store*. ACM SIGOPS Operating Systems Review, 41(6), 205-220.
- [12] Das, A., Gupta, I., & Motivala, A. (2002). *SWIM: Scalable weakly-consistent infection-style process group membership protocol*. IEEE/IFIP International Conference on Dependable Systems and Networks, 303-312.
- [13] Ongaro, D., & Ousterhout, J. (2014). *In search of an understandable consensus algorithm (extended version)*. USENIX Annual Technical Conference, 305-319.
- [14] Cheshire, S., & Krochmal, M. (2013). *Multicast DNS* (RFC 6762). IETF.
- [15] Cheshire, S., & Krochmal, M. (2013). *DNS-Based Service Discovery* (RFC 6763). IETF.
- [16] Kubernetes Authors. (2026). *Kubernetes Services, Load Balancing, and Networking: Service*. <https://kubernetes.io/docs/concepts/services-networking/service/>
- [17] Kubernetes Authors. (2024). *Kubernetes Network Policies*. <https://kubernetes.io/docs/concepts/services-networking/network-policies/>
- [18] Istio Authors. (2025). *Traffic Management Concepts*. <https://istio.io/latest/docs/concepts/traffic-management/>

- [19] HashiCorp. (2026). *Consul Discover Services Overview*. <https://developer.hashicorp.com/consul/docs/discover>
- [20] etcd Authors. (2025). *etcd API guarantees*. https://etcd.io/docs/v3.6/learning/api_guarantees/
- [21] Al-Shaer, E., & Hamed, H. (2009). *Firewall policy queries*. IEEE Transactions on Parallel and Distributed Systems, 20(6), 766-777.
- [22] Jeffrey, A., & Samak, T. (2009). *Model checking firewall policy configurations*. IEEE International Symposium on Policies for Distributed Systems and Networks, 60-67.